

"XGBoost"

XGBoost → not an algorithm it's a library build Date:

based on Gradient Boost. why GB?

XGBoost Features

Flexibility:

- ① Cross platform → any OS
- ② Multiple language
- ③ can be used like ⇒ train using python and same trained obj can be used in java or more.
- ④ Integrated with other ML libraries.
- ⑤ Support all type of ML problem.

- ① Flexibility → cause we can use any differentiable cost/loss function.
- ② Performance
- ③ Robust → can be used for classification and Reg.
can handle missing values.

Speed:

- ① Parallel Processing
- ② Optimized Data Structure
- ③ Cache Awareness
- ④ Out of core computing.
- ⑤ Distributed computing
- ⑥ GPU support

Boosting is a sequential method, Parallel not for model training. Basically build the DT of a stage parallelly

use column block
store data columnwise
instead of rowwise

parallelly calculate gini
or entropy for all
feature

For numerical store as
histogram and store
bin in cache

Performance:

- ① Regularized Learning Objective (to avoid overfitting)
- ② Handle missing values → send missing value in left and right child of tree and take where impurity gain max
- ③ Sparsity Aware Split Finding
- ④ Efficient split finding (weighted Quantile Sketch + Approximate Tree Learning)
divided data into bin based on quantile value and find splitting value from the bin.
- ⑤ Tree Pruning

"XGBoost for Regression"

① start with a base model (mean)

② Train Decision Tree and predict residuals.

* In Gradient Descent we use vanilla DT → Based on Gini/Entropy

* xgboost tree is different. (use different criteria)

Note working flow of xgboost is same as Gradient Boost but Decision tree construction is different.

i/p cgpa	o/p Package	mean model stage 1	residual res1
6.7	4.5	7.4	-2.9
9	11	7.4	3.6
7.5	6	7.4	-1.4
5	8	7.4	0.6

after calculating Decision Tree and for each leaf we pass i/p and residual to train a **Similarity Score** → calculate ss for all leaf node.

let $\lambda = 0$

$$SS = \frac{(-2.9 + 3.6 + 0.6 - 1.4)^2}{4}$$

$$= 0.0025$$

$$SS = \frac{(\sum \text{residual})^2}{\text{num of residual} + \lambda}$$

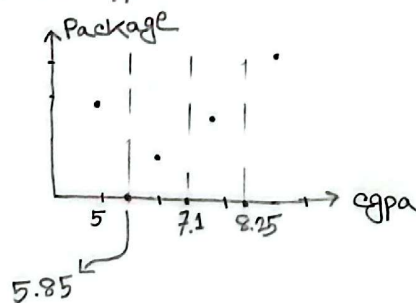
regularization parameter

now train a DT using cgpa and res1

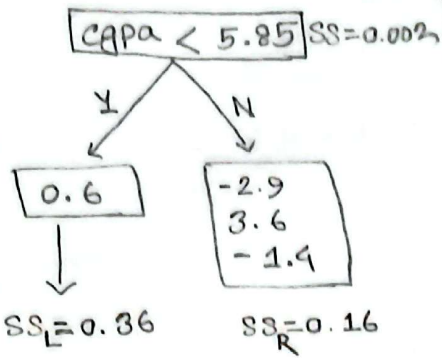
① find the split value → where

To find split value sort the i/p

5 } → mean → 5.85
6.7 } → " → 7.1
7.5 } → " → 8.25
9 }

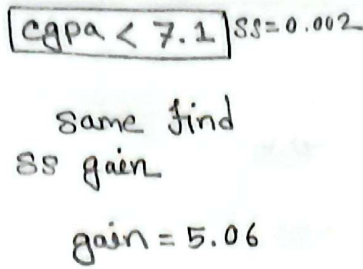


split on 5.85

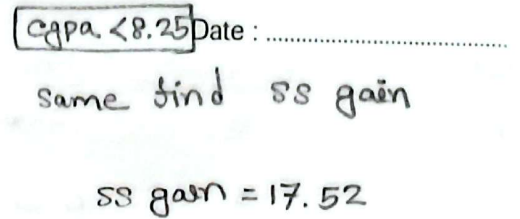


$$SS \text{ gain} = SS_L + SS_R - SS_{\text{root}} = 0.518$$

split on 7.1



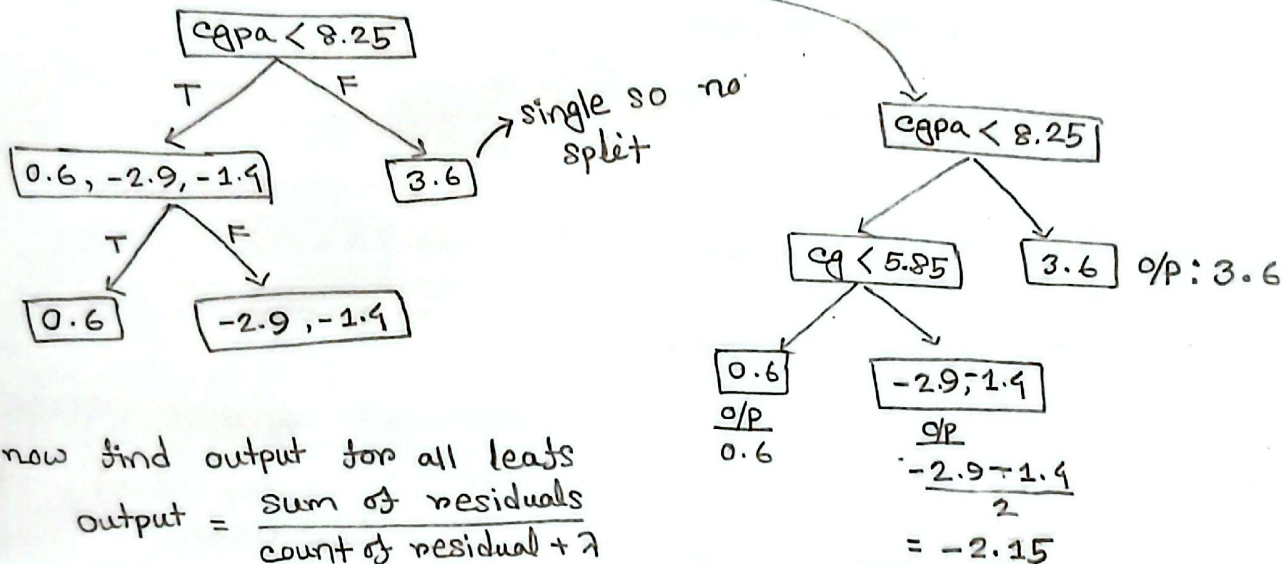
split on 8.25



* Take that split where gain maximum

First decision Tree

Note xgboost default tree depth 6



now find output for all leafs
 output = $\frac{\text{sum of residuals}}{\text{count of residual} + 1}$

now find residual-2 and train stage-3 decision tree and so on.



Oni

XGBoost for Classification

<u>cgpa</u>	<u>placement</u>	<u>Pred1</u>	<u>res1</u>
5.70	0	0.6	-0.65
6.25	1	0.6	0.4
7.15	0	0.6	-0.6
8.10	1	0.6	0.4
9.60	1	0.6	0.4

Workflow same as gradient boosting

- ① Take a log-odd model
- ② find residual
- ③ Train DT Regressor to predict residual

$$\log(\text{odds}) = \log_e \left(\frac{P}{1-P} \right) \quad \text{where } P \text{ is +ve class}$$

stage-1 $\log(\text{odd}) = \log_e \left(\frac{3/5}{1 - 3/5} \right) = 0.405$

$$\text{Probability} = \frac{e^{0.405}}{1 + e^{0.405}} = 0.6$$

stage-2 Fit a DT using cgpa and res1

- ① Sort the feature in ascending
- ② Form a leaf node and add all residual to this leaf node.
- ③ find similarity score

$$SS = \frac{(\sum \text{res}_i)^2}{\sum \{P(1-P)\} + \lambda}$$

$P \rightarrow$ Previous probability
 $\downarrow$
 Probability that Previous model Predicted.

SS = 0

- ④ now find splitting points and split where gain is high.

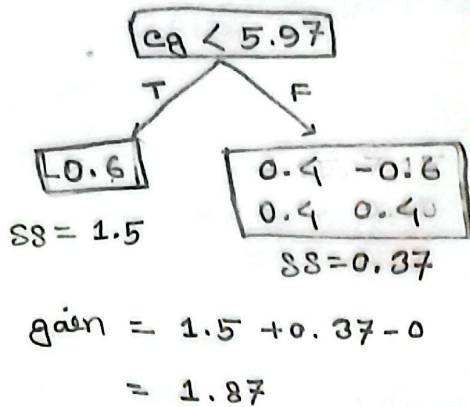
cgpa (sorted)

5.70	5.97
6.25	6.07
7.15	7.62
8.10	8.87
9.60	

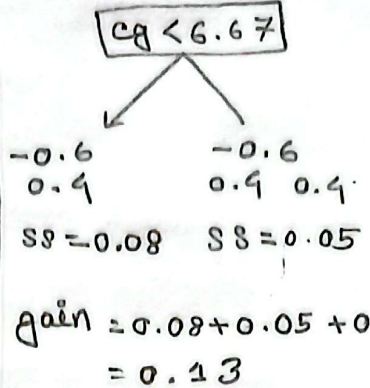
$-0.6, 0.4, 0.6, 0.4, 0.4$ ($SS=0$)

Date :

split on 5.97



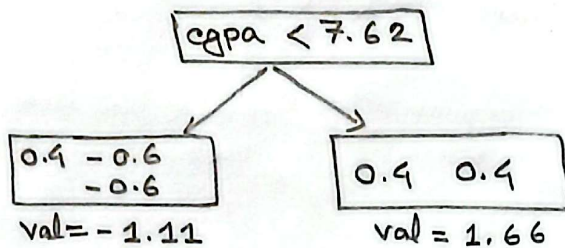
split on 6.67



$cg < 7.62$
gain = 2.22

highest gain 2.22 for $cgpa < 7.62$

For DT-1



After finding the tree calculate value for each leaf -

$$\text{leaf value} = \frac{\sum \text{residual}(i)}{\sum \{P_i (1 - P_i)\} + 2} \quad P_i \rightarrow \text{Probability got from prev stage}$$

now do the Prediction for Tree-1 and also find residual.

cgpa	placed	Pred2 (log-odd)	p
5.70	0	$0.405 + 0.3 * (-1.11) = 0.072$	0.518
6.25	1	0.072	0.518
7.15	0	0.072	0.518
8.10	1	0.903	0.712
9.60	1	0.903	0.712

$$\text{Pred} = m_1 + \ln * m_2$$

↑
0.3 → default

$$\text{res2} = \{\text{placed} - p\}$$

-0.518
0.482
-0.518
0.282
0.282

now find Probability and then residual



Regression

$$\text{Similarity Score} = \frac{(\sum \text{residuals})^2}{N + \lambda}$$

$$\text{output value} = \frac{\sum \text{Residuals}}{N + \lambda}$$

leaf weight

$$\text{Prediction} = F_0 + \sum_{i=1}^M \eta F_i(x)$$

Decision Tree

For a tree we have to calculate the value of each leaf using leaf weight formula

objective function

XGBoost loss function

$$L = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

where $\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \|w\|^2$

regularization term

num of leaf node in k-th Tree

leaf weight

regularization term

* we want the value of w so that the loss function became minimum

Step-1

For a single model [constant model]

$$L^{(1)} = \sum_{i=1}^n L(y_i, \hat{y})$$

Step-2

For 2 model [1 constant model + 1 Tree]

$$L^{(2)} = \sum_{i=1}^n \left[L(y_i, f_1(x_i) + f_2(x_i)) + \Omega(f_2(x_i)) \right]$$

→ Ω only applied on Tree model

Step-3

now add another tree model to previous one

$$L^{(3)} = \sum_{i=1}^n \left[L(y_i, f_1(x_i) + f_2(x_i) + f_3(x_i)) + \Omega(f_3(x_i)) \right]$$

regularization only applied on model-3 cause we already added regularization on model-2 in previous step.

At step-t objective function will be

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \left[L(\theta_i, \hat{y}_1(x_i) + \hat{y}_2(x_i) + \dots + \hat{y}_t(x_i)) + \lambda \hat{y}_t(x_i) \right] \quad \text{Date : } \dots\dots\dots$$

$$= \sum_{i=1}^n \left[L(\theta_i, \hat{y}_i^{(t-1)} + \hat{y}_t(x_i) + \lambda \hat{y}_t(x_i)) \right] \quad \text{where } \hat{y}_i^{(t-1)} = \hat{y}_1(x_i) + \hat{y}_2(x_i) + \dots + \hat{y}_t(x_i)$$

At any stage-t objective function

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \left[L(\theta_i, \hat{y}_i^{(t-1)} + \hat{y}_t(x_i)) + \lambda \hat{y}_t(x_i) \right] \quad n \rightarrow \text{no of rows in trainset}$$

now find a value for this last tree so that the objective function become minimized.

graph will be



→ Doing differentiation and optimization is quite difficult cause not a smooth curve

so we will replace this function with Taylor Series.

Taylor Series can approximate any complex function with polynomial.

$$f(x) = f(a) + \frac{f'(a)}{1!} (x-a) + \frac{f''(a)}{2!} (x-a)^2 + \frac{f'''(a)}{3!} (x-a)^3 + \dots$$

Applying Taylor series on the objective function (expanding 2nd order)

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \left[L(\theta_i, \hat{y}_i^{(t-1)} + \hat{y}_t(x_i)) \right] + \lambda \hat{y}_t(x_i)$$

[replace x with a and a with its val] → applying Taylor series on this part where

$$f(a) = \sum_{i=1}^n L(\theta_i, \hat{y}_i^{(t-1)}) \rightarrow \text{Part-1}$$

$$x = \hat{y}_i^{(t-1)} + \hat{y}_t(x_i)$$

point of change $a = \hat{y}_i^{(t-1)}$

$$f'(a)(x-a) = \left[\sum_{i=1}^n \frac{d}{d\hat{y}_i^{(t-1)}} L(\theta_i, \hat{y}_i^{(t-1)}) \right] \hat{y}_t(x_i) \rightarrow \text{part 2}$$

$$\frac{f''(a)}{2} (x-a)^2 = \frac{1}{2} \sum_{i=1}^n \frac{d^2}{d(\hat{y}_i^{(t-1)})^2} L(\theta_i, \hat{y}_i^{(t-1)}) \hat{y}_t^2(x_i) \rightarrow \text{Part-3}$$

Oricef®

putting part- 1, 2, 3 together

$$\mathcal{L}^{<t>} \approx \sum_{i=1}^n \left[L(\theta_i, \hat{\theta}_i^{<t-1>}) + \underbrace{\left(\frac{\partial L(\theta_i, \hat{\theta}_i^{<t-1>})}{\partial \hat{\theta}_i^{<t-1>}} \right)}_{\text{this term is called gradient } (g)} \hat{\theta}_t(x_i) + \frac{1}{2} \underbrace{\left(\frac{\partial^2 L(\theta_i, \hat{\theta}_i^{<t-1>})}{\partial (\hat{\theta}_i^{<t-1>})^2} \right)}_{\text{this term is called hessian } (h)} \hat{\theta}_t^2(x_i) \right]$$

$$\mathcal{L}^{<t>} \approx \sum_{i=1}^n \left[L(\theta_i, \hat{\theta}_i^{<t-1>}) + g_i \hat{\theta}_t(x_i) + \frac{1}{2} h_i \hat{\theta}_t^2(x_i) \right] + \underbrace{\lambda \sum_{i=1}^n \hat{\theta}_t(x_i)}_{\text{this term is called } \gamma}$$

our main goal was to find a value of $\hat{\theta}_t(x_i)$ which will minimize $\mathcal{L}$ this function but $L(\theta_i, \hat{\theta}_i^{<t-1>})$ don't have $\hat{\theta}_t(x_i)$ means constant so differentiate value will be zero so remove this.

$$\mathcal{L}^{<t>} = \sum_{i=1}^n \left[g_i \hat{\theta}_t(x_i) + \frac{1}{2} h_i \hat{\theta}_t^2(x_i) \right] + \underbrace{\lambda \sum_{i=1}^n \hat{\theta}_t(x_i)}_{\text{expand this term}}$$

$\lambda \rightarrow$ L2 penalty on least weights
 $\gamma \rightarrow$ penalizes adding a least

$$\mathcal{L}^{<t>} = \sum_{i=1}^n \left[g_i \hat{\theta}_t(x_i) + \frac{1}{2} h_i \hat{\theta}_t^2(x_i) \right] + \gamma^T + \frac{1}{2} \lambda \left[\omega_1^2 + \omega_2^2 + \dots + \omega_T^2 \right]$$

$$= \sum_{i=1}^n \left[g_i \hat{\theta}_t(x_i) + \frac{1}{2} h_i \hat{\theta}_t^2(x_i) \right] + \gamma^T + \frac{1}{2} \lambda \sum_{j=1}^T \omega_j^2$$

as we have 2 $\sum$ with different limit so we have to make the limit common.

$$\mathcal{L}^{<t>} = \sum_{j=1}^T \left[\sum_{i \in I_j} g_i \omega_j + \frac{1}{2} \sum_{i \in I_j} h_i \omega_j^2 \right] + \gamma^T + \frac{1}{2} \lambda \sum_{j=1}^T \omega_j^2$$

Here what we did is: $\hat{\theta}_t(x_i) = \omega_j$ and $\sum_{i=1}^n = \sum_{j=1}^T \sum_{i \in I_j}$

$I_j \rightarrow$ set of these rows which belongs to least j

$$\mathcal{L}^{<t>} = \sum_{j=1}^T \left[\sum_{i \in I_j} g_i \omega_j + \frac{1}{2} \sum_{i \in I_j} h_i \omega_j^2 + \frac{1}{2} \lambda \omega_j^2 \right] + \gamma^T$$

$$\mathcal{L}^{<t>} = \sum_{j=1}^T \left[\sum_{i \in I_j} g_i \omega_j + \frac{1}{2} \omega_j^2 \left(\sum_{i \in I_j} h_i + \lambda \right) \right] + \gamma^T$$

τ is the count of leaf so for only one leaf.

$$L^{(t)} = \sum_{i \in I_j} \theta_i \omega_j + \frac{1}{2} \omega_j^2 \left(\sum_{i \in I_j} h_i + \tau \right) + \gamma \tau$$

Date :

$$\frac{\partial L^{(t)}}{\partial \omega_j} = \sum_{i \in I_j} \theta_i + \omega_j \left(\sum_{i \in I_j} h_i + \tau \right) = 0$$

$$\Rightarrow \omega_j = \frac{- \sum_{i \in I_j} \theta_i}{\sum_{i \in I_j} h_i + \tau}$$

now if we want calculate leaf value of j -th leaf then we have to calculate gradient (θ_i) of these row i which fall into j -th leaf

now we have to find θ_i and h_i

For Regression

$$\theta_i = \frac{\partial L(\hat{y}_i, y_i)}{\partial \hat{y}_i} = \frac{\partial}{\partial \hat{y}_i} \left[\frac{1}{2} \sum (y_i - \hat{y}_i)^2 \right] = - \sum (y_i - \hat{y}_i)$$

$\hat{y}_i$ of previous step

$$\omega_j = - \frac{\sum_{i \in I_j} (y_i - \hat{y}_i^{(t-1)})}{\sum_{i \in I_j} h_i + \tau}$$

- sum of residual

$\sum 1 + \tau$

$y_i - \hat{y}_i^{(t-1)}$ → residual of i -th row which fall into j -th leaf

for hessian derivate twice of L

$$\text{so } \frac{\partial^2 L}{\partial \hat{y}_i^2} = 1$$

$$\omega_j = \frac{\text{sum of residual}}{\text{no of residual} + \tau}$$

now put ω_j value to actual objective function

$$L^{(t)} = \sum_{j=1}^T \left[\left(\sum_{i \in I_j} \theta_i \right) \frac{- \sum_{i \in I_j} \theta_i}{\sum_{i \in I_j} h_i + \tau} + \frac{1}{2} \left(\frac{\left(\sum_{i \in I_j} \theta_i \right)^2}{\left(\sum_{i \in I_j} h_i + \tau \right)^2} \times \left(\sum_{i \in I_j} h_i + \tau \right) \right] + \gamma \tau$$

$$= \sum_{j=1}^T \left[\frac{- \left(\sum_{i \in I_j} \theta_i \right)^2}{\sum_{i \in I_j} h_i + \tau} + \frac{1}{2} \frac{\left(\sum_{i \in I_j} \theta_i \right)^2}{\sum_{i \in I_j} h_i + \tau} \right] + \gamma \tau$$

$$\mathcal{L}^{(t)} = \sum_{j=1}^T \left[-\frac{1}{2} \frac{\left(\sum_{i \in I_j} \theta_i \right)^2}{\sum_{i \in I_j} h + \lambda} \right] + \gamma T \quad \xrightarrow{\text{similarity score}} \quad \text{similarity score}$$

$g \rightarrow \text{residual}$

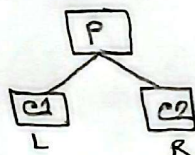
This eqn signifies that at iteration t if we add a new tree then this will be the minimum loss. Also we can use this equation to measure the quality of current tree instead of gini/entropy score.

At t iteration, let's make the tree: $\rightarrow 1 \text{ leaf: } \boxed{T=1}$
 current tree (initially) $\rightarrow \square$

parent $\leftarrow \mathcal{L}^{(t)}(P) = -\frac{1}{2} \frac{\left(\sum_{i \in I_P} \theta_i \right)^2}{\sum_{i \in I_P} h + \lambda} + \gamma$

*add another level

now $\boxed{T=2}$



child $\leftarrow \mathcal{L}^{(t)}(C) = -\frac{1}{2} \frac{\left(\sum_{i \in I_{jL}} \theta_i \right)^2}{\sum_{i \in I_{jL}} h + \lambda} + \frac{-1}{2} \frac{\left(\sum_{i \in I_{jR}} \theta_i \right)^2}{\sum_{i \in I_{jR}} h + \lambda} + 2\gamma$

Loss reduces after adding a new level

$$\mathcal{L}^{(t)}(P) - \mathcal{L}^{(t)}(C) = -P + L + R$$

$$\rightarrow \text{gain} = \frac{1}{2} \left[\frac{\left(\sum_{i \in I_{jL}} \theta_i \right)^2}{\sum_{i \in I_{jL}} h + \lambda} + \frac{\left(\sum_{i \in I_{jR}} \theta_i \right)^2}{\sum_{i \in I_{jR}} h + \lambda} - \frac{\left(\sum_{i \in I_P} \theta_i \right)^2}{\sum_{i \in I_P} h + \lambda} \right] - \gamma$$